

Spark Processing Evidence

HDFS Ingestion

A single-node Hadoop 3.4.1 cluster (no root required, user-local tarball) is brought up and demonstrated with:

```
scripts/setup_hdfs.sh # download, format namenode, start daemons, copy Parquet in
scripts/hdfs_demo.sh  # start an existing cluster and read the data back
```

The processed Parquet tree is put into HDFS and taken back from there to compare instead of being assumed to be done right.

Spark Ingestion Scripts

src/utiq/pipeline/ingest.py reads raw CSV against the declared schemas in schemas.py and writes partitioned Parquet:

```
df.write.mode("overwrite").partitionBy("month")
```

Cleaning Transformations

src/utiq/pipeline/clean.py applies documented correct / quarantine / deduplicate rules, recording the original value, the rule applied, the adjusted value, and the final disposition of all records affected.

Spark Joins and Spark SQL Queries

Integration stage(src/utiq/pipeline/integrate.py) loads 8 clean tables as SparkSQL temporary views and performs a join operation in a single SQL command to create a trip level fact table:

Aggregations

All aggregations at route, stop, and network levels (route performance evaluation, bottlenecks, headway / bunching, ODs) happen in src/utiq/analysis/ using the integrated fact table.

Partition Strategy

The process of partitioning by months is done on only those four tables whose size grows as time progresses (i.e., trips, passenger counts, delays, and tickets), and not on smaller reference tables for which addition of a partition column does not serve any purpose.

Parquet Output

Written in the ingest and integrate stages; can be reread from HDFS in the demo script.

Processing Logs

All stage executions are timed and recorded in SQLite (src/utiq/store.py). The run log now contains 198 stage executions, of 17 different stages, with 17 recorded failures, which are kept intentionally, as failure and fix of a stage are a part of the history of the result. Can be viewed using:

```
utiq status
```

Performance Results

Per-stage timing, most recent run vs. best run recorded:

Stage	Runs	Last Run (s)	Best Run (s)
ml-python	10	656.9	139.1
ml-delay	12	474.8	267.7
clean	10	307.5	99.2
ml-spark	9	285.8	198.4
network	16	169.5	36.0
analytics	23	125.7	15.7
ingest	8	119.0	67.4
quality	8	116.1	41.0
integrate	11	108.1	14.6
catalog	10	95.1	60.2
features	12	94.4	15.1
hdfs	13	55.1	20.1

Spark MLlib Evidence

Classification task: Predicting overcrowding (two classes). Three machine learning models were compared as requested.

Features

- Categories (3): route_id, vehicle_type, day_type
- Numbers (12): direction, dep_hour, capacity, route_reliability, demand_mult, planned_min, route_km, n_stops, is_peak, is_weekend, month, dow

Algorithms, Hyperparameters, and Results

Model	Hyperparameters	Val. AUC	Test AUC	Precision	Recall	F1	Accuracy
LogisticRegression	maxIter=50, regParam=0.0	0.9669	0.9648	0.9134	0.9139	0.9136	0.9139
DecisionTree	maxDepth=8, minInstancesPerNode=1	0.8125	0.8041	0.8554	0.8588	0.8451	0.8588
RandomForest	maxDepth=10, numTrees=60	0.9326	0.9314	0.8329	0.8169	0.7725	0.8169

Confusion Matrix (Test Quarter, Best Model)

	Predicted: Not Overcrowded	Predicted: Overcrowded
Actually Not Overcrowded	111,097	6,300
Actually Overcrowded	6,869	28,643

Miss rate on genuinely overcrowded trips: 19.3% (6,869 of 35,512).

Saved Model, Version, Sample Predictions

Models are serialized into data/models directory with version number incremented, metrics achieved, complete list of features they were fit on, and library version numbers (src/utiq/registry.py). Metrics are also stored in the SQLite application database as the history of the model is now queryable rather than recomputed each time. Predictions for each trip in the test quarter are written to dual_pipeline_comparison.csv file (Section 7 below), one row per trip with its probability and class.